Seminario de Arquitecturas de Software:
Un Análisis Comparativo de Estilos Basados en Capas y Dominio

Ingeniería de Software

Equipo: RAM + L^2

Integrantes y preguntas en las que trabajó cada uno:

- Luis Daniel Friol Rodríguez: pregunta 1.
- Ailema Matos Rodríguez: pregunta 2.
- Lianet Tamarit Tejas: pregunta 3.
- Meylí Jiménez Velázquez: pregunta 4 y pregunta 6 (segunda parte).
- Raúl Roberto Espinosa Poma: pregunta 5 y pregunta 6 (primera parte).

Repositorio del proyecto:
`https://github.com/Charge1500/nutritional_management_system`

1. Respuestas a las preguntas

1.1. Pregunta 1: Análisis comparativo de comunicación y acoplamiento

Un sistema de subastas en línea requiere que, al finalizar una subasta, se realicen múltiples acciones: notificar al ganador, procesar el pago y actualizar el inventario. En un diseño con microservicios, un arquitecto podría optar por una coreografía de eventos (comunicación asíncrona) o una orquestación mediante un servicio mediador. En un diseño con *Service-Based Architecture*, estas acciones probablemente se ejecutarían dentro de un mismo servicio de dominio de grano grueso. Basado en los conceptos de connascencia y los *trade-offs* de la comunicación síncrona vs. asíncrona, analice: (a) ¿cómo afecta cada enfoque al acoplamiento dinámico entre los componentes? (b) ¿qué implicaciones tiene cada diseño para la tolerancia a fallos y la consistencia de los datos (ACID vs. BASE) si el servicio de pago falla justo después de que se notifica al ganador?

1.1. Acoplamiento dinámico según el enfoque

Microservicios con coreografía. Cada servicio reacciona a los eventos que le interesan (SubastaCerrada, PagoProcesado, InventarioActualizado) sin un coordinador central. El acoplamiento dinámico es bajo: ningún servicio conoce el flujo completo ni depende del orden explícito de los demás (Richards y Ford, 2020, cap. 3). El costo es que la relación de dependencia queda implícita en la cadena de eventos: si un servicio no publica el evento esperado, los siguientes no se enteran, y rastrear el flujo es más difícil. Es un connascence de ejecución distribuido y poco visible.

Microservicios con orquestación. Un servicio mediador conoce el flujo y llama a cada servicio en orden. Aquí aparece un connascence de orden y temporización claro: todos los participantes dependen de la secuencia que define el orquestador (Richards y Ford, 2020, cap. 3). La ventaja es visibilidad total y manejo centralizado de errores; la desventaja es mayor acoplamiento y que el orquestador se vuelve un posible punto único de fallo y cuello de botella.

SBA (servicios de grano grueso). Notificar, pagar y actualizar inventario ocurren dentro de un mismo servicio de dominio. No hay llamadas remotas ni saltos de eventos entre esos pasos, así que el acoplamiento dinámico es mínimo. El costo es un acoplamiento estático alto (toda la lógica junta) y menor elasticidad: no se puede escalar solo el paso de pago (Richards y Ford, 2020, cap. 13).

1.2. Si el pago falla después de notificar al ganador

SBA con transacción ACID. Si notificar y pagar están dentro de la misma transacción local, el fallo del pago provoca *rollback*: la transacción no se confirma y el sistema queda coherente. Es la vía más simple para consistencia fuerte, siempre que las operaciones estén en un mismo servicio y una misma base de datos.

Microservicios con coreografía (BASE / consistencia eventual). La notificación ya se envió y no se puede "deshacer". El sistema queda en un estado intermedio y se necesita un evento de compensación (patrón Saga): por ejemplo, emitir SubastaCancelada para avisar al ganador y liberar la reserva de inventario (Ford et al., 2021, caps. 6 y 9). La tolerancia a fallos depende de la idempotencia y los reintentos de cada servicio. Ventaja: los servicios no se bloquean entre sí. Desventaja: la lógica de compensación es compleja y puede fallar ella misma.

Microservicios con orquestación (Saga orquestada). El mediador detecta el fallo del pago y ejecuta explícitamente los pasos de compensación (reembolso, cancelación de la notificación, liberación de inventario). El flujo de compensación es claro y auditable, pero el orquestador es punto único de fallo y la compensación no "restaura" la transacción original: es una nueva operación de acción opuesta.

1.3. *Trade-offs*

Consistencia vs. elasticidad. SBA ofrece consistencia fuerte (ACID) con menor elasticidad; los microservicios ofrecen elasticidad y autonomía a cambio de consistencia eventual y compensaciones.

Visibilidad vs. acoplamiento. La coreografía tiene el acoplamiento más débil pero la menor visibilidad del flujo; la orquestación da control total pero introduce acoplamiento y riesgo de punto único.

Complejidad de compensación. Cuanto más distribuido el flujo, más pasos de compensación hay que diseñar, probar y monitorear.

1.2. Pregunta 2: Diseño para elasticidad extrema y consistencia de datos

Considere el sistema de venta de entradas para conciertos descrito en el libro (capítulo 15). Durante el lanzamiento de entradas para un artista popular, el sistema experimenta un aumento repentino de usuarios de cientos a decenas de miles en segundos. Se le pide, como arquitecto, diseñar la solución: (a) justifique por qué una *Space-Based Architecture* (SBA) sería una elección más adecuada que una *Microservices Architecture* para este escenario, enfocándose en la elasticidad y la escalabilidad; (b) describa cómo mitigar el riesgo de colisiones de datos en cachés replicadas en la venta de asientos numerados, donde la consistencia es crucial, y qué mecanismos o *trade-offs* consideraría.

2.1. Por qué una SBA es más adecuada que microservicios

Naturaleza de la carga. Al crecer la carga de una aplicación web, el cuello de botella se desplaza del servidor web al servidor de aplicación y, por último, a la base de datos, el componente más difícil y caro de escalar (Richards y Ford, 2020, cap. 15; Figura 1). La SBA está diseñada específicamente para problemas de alta escalabilidad, elasticidad y concurrencia, y para aplicaciones con volúmenes de usuarios concurrentes variables e impredecibles; es adecuada para picos de carga y para más de 10 000 usuarios concurrentes, y los sistemas de venta de entradas en línea y de subastas en línea son ejemplos típicos (Richards y Ford, 2020, cap. 15, sección «Implementation Examples»).

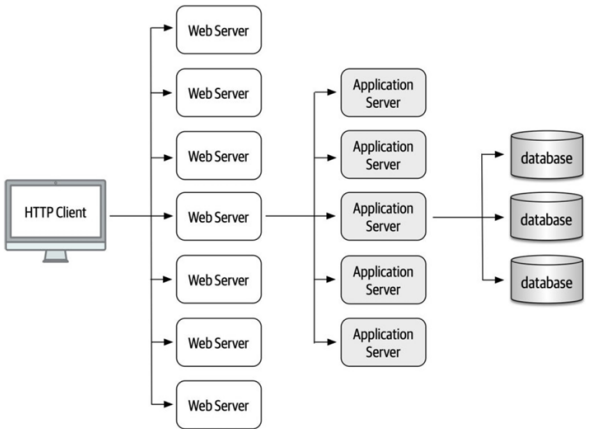


Figure 15-1. Scalability limits within a traditional web-based topology

Figura 1: Límites de escalabilidad en una topología web tradicional. *Nota. Fundamentals of Software Architecture: An Engineering Approach* (fig. 15-1), por M. Richards y N. Ford, 2020, O'Reilly Media.

El sistema de entradas descrito en el libro. El volumen de usuarios es relativamente bajo hasta que se anuncia un concierto popular; al salir a la venta pasa de cientos a miles, posiblemente decenas de miles, y las entradas se agotan en minutos. La disponibilidad de asientos debe actualizarse y ofrecerse lo más rápido posible, y acceder de forma síncrona a una base de datos central difícilmente funcionaría con decenas de miles de peticiones concurrentes (Richards y Ford, 2020, cap. 15).

Cómo logra la SBA la elasticidad y la escalabilidad. La alta escalabilidad, elasticidad y rendimiento se consiguen eliminando la base de datos central como restricción síncrona y usando mallas de datos replicadas en memoria. Los datos viven en memoria y se replican entre las unidades de procesamiento activas; cuando una unidad actualiza datos, los envía de forma asíncrona a la base de datos, normalmente mediante mensajería con colas persistentes (*data pumps* y *data writers*). Las unidades se inician y se detienen dinámicamente según la carga, y al no haber base de datos central en el procesamiento transaccional normal, se elimina ese cuello de botella (Richards y Ford, 2020, cap. 15). En el ejemplo de las entradas, el gestor de despliegue (*deployment manager*) detecta el aumento de usuarios y arranca muchas unidades de procesamiento; lo óptimo es configurarlo para que las tenga listas poco antes de la salida a la venta (Richards y Ford, 2020, cap. 15). La SBA recibe cinco estrellas en elasticidad, escalabilidad y rendimiento, que son sus atributos principales (Richards y Ford, 2020, cap. 15, fig. 15-15).

Por qué los microservicios encajan peor aquí. Los microservicios destacan en escalabilidad y elasticidad (Richards y Ford, 2020, cap. 17, fig. 17-13), de modo que la ventaja de la SBA en este escenario no es que los microservicios no escalen, sino lo siguiente:

- **Rendimiento.** El rendimiento suele ser un problema en microservicios: hay que hacer muchas llamadas de red para completar el trabajo y verificar identidad y acceso en cada *endpoint*, lo que implica una sobrecarga alta (Richards y Ford, 2020, cap. 17). La SBA, en cambio, atiende cada petición con datos en memoria de la unidad.
- **La base de datos sigue en el camino crítico** (análisis propio). Cada microservicio posee su base de datos y el flujo de compra (consultar disponibilidad, reservar, cobrar) la consulta de forma síncrona; escalar la capa de aplicación sirve de poco si el almacenamiento se satura en segundos.

- **Cadena de servicios** (análisis propio). Una compra atraviesa varios servicios y, bajo un pico, todos los de la cadena deben escalar a la vez, mientras que las unidades de procesamiento de la SBA son autosuficientes.

Aspecto	**Space-Based**	**Microservicios**
Elasticidad y escalabilidad	Cinco estrellas (fig. 15-15)	Altas (fig. 17-13)
Rendimiento	Cinco estrellas; sin base de datos central en el camino transaccional	A menudo un problema por las llamadas de red y las verificaciones de seguridad
Camino de una petición	Memoria local de la unidad de procesamiento; persistencia asíncrona	Varios servicios por red y su base de datos
Consistencia	Eventual respecto a la base de datos (hay que gestionarla)	Fuerte dentro de cada servicio, eventual entre servicios
Complejidad, pruebas y coste	Muy compleja, pruebas con una estrella y coste relativamente alto	Compleja; muy buena capacidad de prueba

Cuadro 1: Comparación para el escenario de la venta de entradas (elaboración propia a partir de Richards y Ford, 2020, caps. 15 y 17).

Contrapartida. La SBA es un estilo muy complicado por el uso de cachés y la consistencia eventual del almacén principal de datos; las pruebas reciben una estrella porque simular cientos de miles de usuarios concurrentes es complejo y caro, por lo que gran parte de las pruebas de alto volumen se hace en producción; y es relativamente costosa, sobre todo por las licencias de los productos de caché y el alto uso de recursos (Richards y Ford, 2020, cap. 15). Aun así, para un pico extremo y breve como el descrito, esas desventajas son aceptables.

2.2. Mitigación de colisiones de datos en la venta de asientos numerados

El problema. Con cachés replicadas en modo activo/activo, cada unidad de procesamiento puede actualizar su copia de la misma caché con nombre. Una *colisión de datos* ocurre cuando un dato se actualiza en una caché (A) y, durante la replicación hacia otra (B), esa misma caché lo actualiza también: la actualización local de B se sobrescribe con el dato antiguo de A y viceversa, y ambas cachés quedan incorrectas y desincronizadas (Richards y Ford, 2020, cap. 15, sección «Data Collisions»). Un ejemplo es un inventario de 500 unidades que dos instancias actualizan a la vez (Richards y Ford, 2020, cap. 15). En la venta de asientos numerados, el equivalente sería que dos unidades marquen el mismo asiento como vendido a compradores distintos, y el asiento más solicitado es un dato caliente actualizado por muchos usuarios a la vez.

Estimación del riesgo. Las colisiones pueden estimarse probabilísticamente con la fórmula

$$\text{Tasa de colisión} \;=\; N \times \frac{UR^2}{S} \times RL,$$

donde N es el número de instancias que usan la misma caché con nombre, UR la tasa de actualización, S el tamaño de la caché (en filas) y RL la latencia de replicación del producto de caché (Richards y Ford, 2020, cap. 15). Con 20 actualizaciones por segundo, 5 instancias, 50 000 filas y 100 ms de latencia, se obtienen 14,4 colisiones por hora sobre 72 000 actualizaciones (0,02 %) (Richards y Ford, 2020, cap. 15). Reducir la latencia a 1 ms baja la cifra a 0,1 por hora; reducir las instancias de 5 a 2, a 5,8 por hora; y reducir la caché a 10 000 filas la sube a 72 por hora. S es el único factor inversamente proporcional (Richards y Ford, 2020, cap. 15). Conviene calcular

las tasas mínima, normal y pico usando la tasa máxima de actualización en el pico de uso, que aquí corresponde al lanzamiento de entradas (Richards y Ford, 2020, cap. 15).

Criterio	**Caché replicada**	**Caché distribuida**
Optimización	Rendimiento	Consistencia
Tamaño de la caché	Pequeña (<100 MB)	Grande (>500 MB)
Tipo de datos	Relativamente estáticos	Muy dinámicos
Frecuencia de actualización	Relativamente baja	Alta
Tolerancia a fallos	Alta	Baja

Cuadro 2: Caché distribuida frente a replicada. *Nota.* Adaptado de *Fundamentals of Software Architecture: An Engineering Approach* (tabla 15-1), por M. Richards y N. Ford, 2020, O'Reilly Media.

Mecanismos basados en el libro.

1. **Caché distribuida para el estado de los asientos y replicada para el resto.** Los datos que requieren alta consistencia, como los inventarios de productos, suelen justificar una caché distribuida, mientras que los datos que cambian poco (códigos, descripciones) se sirven mejor con una replicada; conviene usar cada modelo según sus fortalezas en lugar de uno solo para toda la aplicación (Richards y Ford, 2020, cap. 15). Aquí, el estado de cada asiento iría en una caché distribuida y los datos estáticos (catálogo de conciertos, descripción del recinto) en cachés replicadas (Figuras 2 y 2). *Trade-offs*: el acceso remoto añade latencia y reduce el rendimiento; si el servidor de caché cae, las unidades quedan inoperantes; y reflejar (*mirroring*) la caché mitiga ese fallo, pero puede producir inconsistencias si el servidor principal cae antes de que el dato llegue al espejo (Richards y Ford, 2020, cap. 15).

2. **Reducir los factores de la fórmula en los datos que sigan replicados.** Menos instancias que compartan la misma caché, mayor tamaño de la caché y menor latencia de replicación disminuyen las colisiones. La latencia depende de la red y de la distancia física entre unidades y debe medirse en producción; si no se dispone del valor real, se sugiere 100 ms como valor de planificación (Richards y Ford, 2020, cap. 15). El coste es más memoria por unidad y productos de caché más rápidos, que son más caros.

3. **No usar *near-cache*.** Las cachés frontales de cada unidad no se sincronizan entre sí, de modo que unidades con el mismo contexto de datos tendrán datos distintos; por eso no se recomienda este modelo en la SBA (Richards y Ford, 2020, cap. 15).

4. **Unidades de procesamiento dedicadas por partición.** En el ejemplo de las subastas, se dedican unidades de procesamiento individuales a cada subasta para asegurar la consistencia de los datos de las pujas (Richards y Ford, 2020, cap. 15). Por analogía, se propone dedicar unidades a cada concierto o sección, de modo que cada asiento tenga un único propietario y no pueda colisionar. *Trade-off*: menos reparto uniforme de la carga y riesgo de una partición "caliente" (extensión propia del ejemplo del libro).

5. **Persistencia asíncrona sin pérdida de datos.** Las actualizaciones llegan a la base de datos mediante *data pumps* y *data writers*, y debe evitarse la pérdida de datos si falla alguna de las muchas piezas móviles (Richards y Ford, 2020, caps. 14 y 15). Se añade una restricción de unicidad sobre (evento, asiento) en la base de datos como última línea de defensa: si una colisión escapara, el *data writer* rechaza la venta duplicada y se dispara una compensación (reembolso o reubicación). Coste: compensaciones posteriores en casos raros.

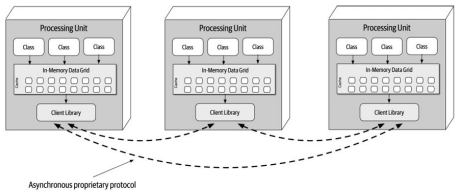


Figure 15-12. Replicated caching between processing units

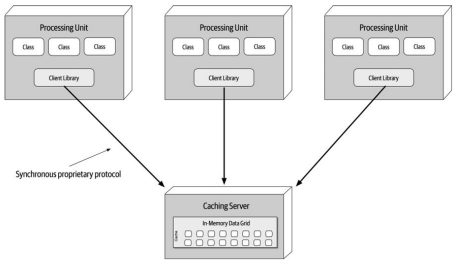


Figure 15-13. Distributed caching between processing units

Figura 2: Caché replicada (izquierda) y caché distribuida (derecha) entre unidades de procesamiento. *Nota. Fundamentals of Software Architecture: An Engineering Approach* (figs. 15-12 y 15-13), por M. Richards y N. Ford, 2020, O'Reilly Media.

Mecanismos complementarios (propuesta propia).

6. **Control de concurrencia optimista.** El asiento lleva un número de versión y la reserva es una operación atómica de comparar y asignar ("si está libre, pasa a retenido"). Si la versión cambió, la segunda petición se rechaza de inmediato. Coste: algunos usuarios reciben rechazos y deben reintentar.

7. **Estados de asiento con retención temporal.** El asiento pasa de *libre* a *retenido* (con tiempo de expiración) y luego a *vendido*. Evita bloqueos largos durante el pago y devuelve el asiento al inventario si el usuario no completa la compra. Coste: asientos inmovilizados temporalmente y lógica de expiración.

8. **Cola de admisión (sala de espera virtual).** Un componente adicional regula cuántas peticiones por segundo llegan a las unidades, lo que reduce UR, el factor de mayor peso en la fórmula, al estar elevado al cuadrado. Coste: esperas para el usuario.

9. **Función de aptitud (*fitness function*).** Una prueba automatizada verifica que nunca existan dos ventas confirmadas del mismo asiento y que la tasa de colisiones calculada con la fórmula se mantenga bajo un umbral al cambiar el número de instancias o la configuración de replicación (Richards y Ford, 2020, cap. 6).

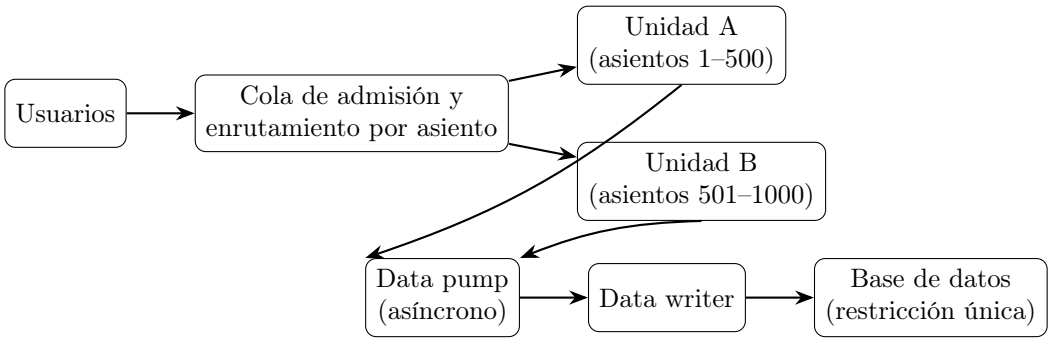


Figura 3: Propuesta de diseño: cada asiento tiene una única unidad propietaria y la persistencia es asíncrona. El enrutamiento por asiento es una variante propia; en el libro la malla de mensajería reparte las peticiones con algoritmos como *round-robin* o *next-available* (elaboración propia a partir de Richards y Ford, 2020, cap. 15).

Mecanismo	**Beneficio**	**Costo / *trade-off***
Caché distribuida para asientos	Consistencia fuerte del estado	Latencia remota; punto único de fallo salvo espejo
Ajustar N, S y RL	Menos colisiones en datos replicados	Más memoria y productos de caché más caros
Unidades por partición	Un solo escritor por asiento	Reparto de carga menos uniforme; partición caliente
Control optimista (versión)	Detecta conflictos sin bloqueos largos	Rechazos y reintentos
Retención con expiración	Evita bloqueos durante el pago	Asientos inmovilizados temporalmente
Cola de admisión	Reduce UR	Esperas; componente adicional
Restricción única en la BD	Integridad final garantizada	Compensaciones en casos raros

Cuadro 3: Resumen de mecanismos de mitigación y sus compromisos.

Trade-offs globales. Cada mecanismo que aumenta la consistencia cede parte de lo que da la SBA: rendimiento (la caché distribuida es más lenta que la replicada) y tolerancia a fallos. El teorema CAP (Gilbert y Lynch, 2002) formaliza que, ante particiones de red, hay que elegir entre consistencia y disponibilidad; se propone elegir consistencia para el estado de los asientos y rendimiento y tolerancia a fallos para los datos de referencia. En zonas sin asiento asignado, el riesgo de colisión es de menor impacto y podría tolerarse consistencia eventual con compensación posterior. La decisión depende del impacto de negocio de un error frente al coste de la latencia añadida.

Para un pico extremo e impredecible como el lanzamiento de entradas, la SBA es más adecuada porque elimina la base de datos como restricción síncrona y permite que el gestor de despliegue arranque unidades autosuficientes antes y durante el pico; los microservicios escalan, pero arrastran la sobrecarga de red y de bases de datos por servicio. Su principal riesgo, las colisiones en cachés replicadas, se controla combinando una caché distribuida para el estado de los asientos, unidades dedicadas por partición, control optimista, retención temporal y una restricción de unicidad final, aceptando latencia, complejidad y menor tolerancia a fallos en esa parte del sistema.

1.3. Pregunta 3: Análisis de fallos y resiliencia en arquitecturas dirigidas por eventos

En una *Event-Driven Architecture* con topología de *broker*, un procesador de eventos (*event processor*) falla repetidamente al procesar un tipo específico de mensaje debido a un error en los datos. Describa el impacto de este fallo en el flujo general del sistema y en la experiencia del usuario final, considerando que no hay un mediador central. Proponga una solución arquitectónica utilizando el *Workflow Event Pattern* descrito en el capítulo 14. Explique detalladamente cómo este patrón mitiga el problema y qué componentes adicionales (como el *workflow processor* y el *dashboard*) serían necesarios, así como los *trade-offs* que introduce (p. ej., procesamiento fuera de orden).

3.1. Impacto del fallo en una topología *broker*

Usemos como ejemplo una tienda en línea: el evento PedidoCreado pasa por el procesador de pagos, luego por el de inventario y luego por el de envíos. El procesador de pagos falla una y otra vez con un mensaje que tiene un dato corrupto (por ejemplo, un monto nulo).

Impacto en el flujo del sistema.

- **Nadie coordina el flujo completo.** En la topología *broker* cada procesador solo conoce el evento que escucha y el que publica. No hay un mediador que sepa en qué paso va cada pedido, así que nadie detecta que ese flujo quedó detenido (Richards y Ford, 2020, cap. 14).
- **El flujo queda cortado a la mitad.** El evento que debía producir el procesador de pagos nunca se publica, así que los procesadores siguientes (inventario, envíos) no se activan para ese pedido. El resto de los mensajes puede seguir procesándose con normalidad, por lo que el sistema parece sano mientras ese pedido queda en un estado intermedio.
- **El mensaje defectuoso se reintenta sin fin.** Si el *broker* lo vuelve a entregar y el procesador vuelve a fallar, se gastan recursos y se pueden acumular mensajes en el canal.
- **Hay inconsistencia entre procesadores.** Si otros procesadores sí reaccionaron al mismo evento (por ejemplo, el de notificaciones envió un correo de confirmación), el sistema queda con datos que no concuerdan entre sí.

Impacto en el usuario final.

- Como la comunicación es asíncrona, el usuario recibe de inmediato una confirmación de que su pedido fue recibido, pero el pedido nunca avanza.
- No recibe ningún error ni explicación, porque nadie tiene la responsabilidad de avisarle.
- Si hay un cobro parcial o una reserva de inventario sin completar, puede quedar un cobro sin envío.

3.2. Solución: *Workflow Event Pattern*

Este patrón, que el libro también presenta como una forma de manejar errores en arquitecturas reactivas, agrega un procesador especializado en recibir los eventos con error en lugar de dejar que el procesador original los reintente indefinidamente (Richards y Ford, 2020, cap. 14).

Cómo funciona, paso a paso.

- El procesador de pagos recibe el mensaje defectuoso y falla.
- En lugar de reintentar o descartarlo, **delega el mensaje a un *workflow processor*** mediante un canal de eventos de flujo de trabajo y continúa con los siguientes mensajes.
- El *workflow processor* analiza el mensaje y **intenta corregir el error de forma programática** (por ejemplo, completar un dato a partir de otra fuente).
- Si no puede corregirlo, **lo envía a un *dashboard*** donde una persona (soporte o un administrador) revisa el dato, lo arregla manualmente y lo marca como corregido.
- El mensaje corregido se **reenvía al canal original**, y el procesador de pagos lo procesa como si fuera el evento inicial.

Componentes adicionales necesarios.

- ***Workflow processor*:** recibe los mensajes con error, intenta repararlos y decide si los reenvía o los escala.
- **Canal (cola) de eventos de flujo de trabajo:** recibe los mensajes con error, separados del flujo normal.

- ***Dashboard*:** interfaz para que una persona vea los mensajes fallidos, los corrija y los reenvíe.
- **Almacenamiento de los mensajes con error y su estado,** para tener trazabilidad de qué falló, qué se corrigió y qué sigue pendiente.

Cómo mitiga el problema.

- El procesador deja de bloquearse en un mensaje defectuoso, así que **el resto del tráfico sigue fluyendo**.
- El error **deja de ser invisible**, porque queda registrado y visible en el *dashboard*.
- El mensaje **no se pierde**: se corrige y se reintegra al flujo.
- El procesador original queda **más simple**, porque no tiene que implementar lógica de reintentos ni de reparación.

3.3. *Trade-offs*

- **Procesamiento fuera de orden.** Mientras el mensaje con error se repara, los mensajes posteriores ya se procesaron. Cuando el corregido vuelve al flujo, se procesa después de otros que en teoría iban después. Si el orden importa (por ejemplo, operaciones sobre un mismo cliente o cuenta), esto puede causar inconsistencias y obliga a diseñar con tolerancia a este desorden.
- **Mayor complejidad.** Hay más componentes que construir, desplegar, monitorear y mantener.
- **Consistencia eventual.** Entre que ocurre el fallo y se corrige, el sistema está en un estado intermedio que el usuario puede ver.
- **Intervención manual.** Los casos que no se corrigen solos dependen de una persona, lo que introduce costos y retrasos.
- **Sigue sin haber un coordinador global.** El patrón maneja los errores, pero no devuelve la visibilidad de un mediador sobre todo el flujo. Para flujos muy complejos o con muchas dependencias, la topología *mediator* puede ser más adecuada, a cambio de menor desacoplamiento y escalabilidad.

1.4. Pregunta 4: El dilema de la granularidad y el acoplamiento en microservicios

Enunciado. Un equipo de desarrollo está migrando un sistema monolítico de gestión de recursos humanos a microservicios. Inicialmente crean servicios muy pequeños para cada entidad (Empleado, Departamento, Nómina, Beneficios). Pronto descubren que completar una operación simple, como contratar a un nuevo empleado, requiere una compleja coreografía de llamadas síncronas entre casi todos los servicios, lo que resulta en un alto acoplamiento y una fragilidad similar a la del monolito original. Basado en los principios de *bounded context* y las guías de granularidad del capítulo 17: (a) ¿cuál fue el error fundamental en el diseño de los servicios? (b) Proponga una nueva estrategia de descomposición. ¿Cómo aplicaría los conceptos de propósito, transacciones y coreografía para redefinir los límites de los servicios y reducir el acoplamiento? Proporcione un ejemplo de cómo agruparía los servicios para la operación de "contratación".

4.1. Error fundamental en el diseño de los servicios

El error fundamental fue crear una granularidad excesivamente pequeña a partir de las entidades individuales Empleado, Departamento, Nómina y Beneficios, en lugar de identificar capacidades de negocio, contextos delimitados y límites transaccionales coherentes. El equipo confundió la independencia de despliegue de componentes pequeños con la autonomía funcional, o sea; distribuyó los componentes pero dejó el comportamiento de contratación dividido entre servicios que necesitan colaborar constantemente para completar una misma operación.

El equipo erróneamente adoptó los nombres (por ejemplo Empleado o Departamento) como límites suficientes sin estudiar el comportamiento de contratación. Que una entidad tenga identidad propia en el modelo de datos no demuestra que sea una capacidad de negocio autónoma. Como se vió en el capítulo 17, el concepto de *bounded context* no equivale a crear un servicio independiente para cada clase o tabla. Los autores explican que un servicio modela un dominio o un flujo de trabajo e incorpora las clases, los componentes y los esquemas de datos necesarios para operar dentro de ese ámbito. Por tanto, un contexto puede tener varias entidades que cooperan para realizar un comportamiento de negocio (Richards y Ford, 2020).

El propósito de este aislamiento es permitir que cada contexto defina los elementos que necesita sin adaptar constantemente su modelo interno a las necesidades de otros contextos. No significa que los servicios no puedan comunicarse porque lo que pide es que esa colaboración no debería exigir compartir sus clases o esquemas internos (Richards y Ford, 2020).

En este capítulo también advierten que el término "microservicio" no debe interpretarse como una obligación de minimizar el tamaño de cada componente, ya que algunos procesos de negocio tienen límites naturales más amplios que otros. En el sistema de este equipo, la contratación es precisamente la prueba de esa fragmentación porque una operación requiere llamadas síncronas entre casi todos los servicios. La separación física no ha eliminado la dependencia funcional, lo que ha hecho es trasladarla a la red. La consecuencia de ésto es que las llamadas de red implican un coste superior al de las llamadas locales y una comunicación excesiva entre servicios puede perjudicar la fiabilidad y la tolerancia a fallos (Richards y Ford, 2020).

Los autores señalan que la atomicidad sencilla de una aplicación monolítica se convierte en un problema cuando intervienen servicios y bases de datos independientes (Richards y Ford, 2020). Por tanto, en este sistema la dificultad se agrava si contratar exige que los cambios en varios servicios se confirmen (o se rechacen) como una unidad. En este caso, el equipo ha dividido una necesidad de consistencia entre límites de servicio distintos y debe introducir coordinación distribuida para conservarla.

4.2. Nueva estrategia de descomposición

La nueva estrategia debe organizar los servicios alrededor de capacidades de negocio cohesionadas y no reproducir directamente las entidades del sistema monolítico. Para la contratación, se propone identificar un núcleo de Gestión de Contratación que concentre las reglas y los datos necesarios para registrar válidamente una relación laboral, manteniendo separados los procesos de Nómina y Beneficios cuando puedan trabajar con autonomía y terminar su trabajo después del alta.

Concepto de propósito. El capítulo 17 añade que idealmente, cada microservicio debería tener una cohesión funcional extremadamente alta y aportar un comportamiento significativo al conjunto de la aplicación (Richards y Ford, 2020). En consecuencia, el propósito de un servicio debería expresarse como una capacidad reconocible del negocio, por ejemplo, "formalizar la contratación de una persona".

Gestión de Contratación podría responsabilizarse de registrar al empleado, formalizar la relación laboral, establecer la fecha de incorporación y registrar la asignación organizativa necesaria

para el alta. Estas responsabilidades se agruparían porque contribuyen a un mismo resultado de negocio (no porque todas pertenezcan genéricamente al área de recursos humanos).

Concepto de transacciones. El libro señala que las entidades que necesitan cooperar en una transacción suelen mostrar a los arquitectos un buen límite de servicio (Richards y Ford, 2020). Por tanto se debe identificar qué cambios deben realizarse conjuntamente para que la contratación sea válida y mantenerlos (cuando sea apropiado) dentro del mismo límite de servicio y de persistencia.

Por ejemplo, si las reglas establecen que no puede existir un alta válida sin un registro de empleado, una relación laboral y una asignación organizativa inicial, esos elementos podrían guardarse con una transacción local en Gestión de Contratación. Pero si crear el perfil de nómina o iniciar la inscripción en beneficios puede ocurrir después sin invalidar el alta, esas tareas no necesitan incorporarse a la misma transacción. La estructura organizativa completa podría mantenerse en otro contexto, mientras Contratación conserva la referencia y los datos que necesita para su propio comportamiento.

Concepto de coreografía. Se puede entender por lo visto en el capítulo que si un arquitecto construye un conjunto de servicios que ofrecen un excelente aislamiento del dominio, pero requieren una comunicación extensa para funcionar, puede considerar volver a agruparlos en un servicio mayor para evitar la sobrecarga de comunicación.

Por ello, la primera medida corresponde a revisar si los participantes deberían estar separados, en vez de sustituir mecánicamente todas las llamadas síncronas por mensajes. Esto se debe a que cambiar el mecanismo de comunicación no corrige por sí mismo una fragmentación incorrecta del comportamiento. Después de corregir esos límites, la coreografía puede usarse para coordinar capacidades autónomas (Richards y Ford, 2020).

4.3. Ejemplo de agrupación de los servicios para la operación de contratación

Un ejemplo de cómo agruparía los servicios para la operación de contratación sería el siguiente (suponiendo que el alta laboral puede confirmarse antes de terminar la preparación de nómina y beneficios):

Servicio propuesto	**Responsabilidad**	**Relación con la contratación**
Gestión de Contratación	Registrar empleado, relación laboral y asignación inicial necesaria	Confirma conjuntamente los cambios que hacen válida el alta
Nómina	Gestionar los datos y las reglas necesarios para el procesamiento salarial	Recibe el evento de contratación y prepara su propio registro
Beneficios	Gestionar elegibilidad e inscripción en prestaciones	Recibe el evento e inicia su proceso según sus reglas
Estructura Organizativa (si es una capacidad autónoma)	Administrar departamentos y su estructura	Brinda la información organizativa requerida (sin compartir directamente su esquema de datos)

Se tendría además un flujo conceptual que separa el resultado inmediato (acta laboral) de sus consecuencias posteriores. Como no garantiza una actualización atómica de todos los servicios, admitiría que Nómina y Beneficios completen su trabajo después. Este es el flujo pensado:

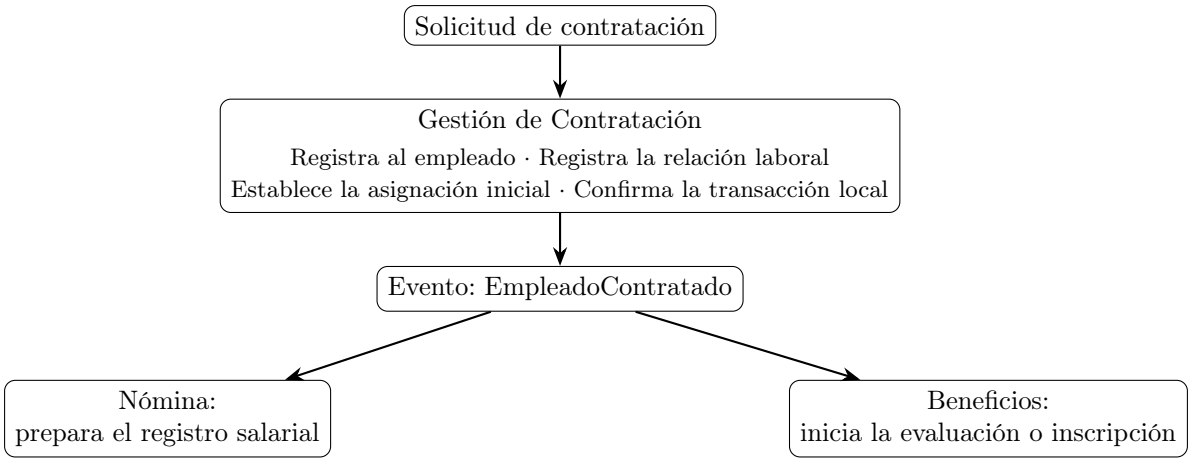


Figura 4: Flujo conceptual de la contratación con coreografía de eventos.

Para concluir, en este primer intento no debe asumirse una descomposición perfecta ya que como se vio destacado en el capítulo: la granularidad, las dependencias de datos y los estilos de comunicación se refinan iterativamente.

1.5. Pregunta 5: Comparación de atributos de calidad y *trade-offs*

Un director de tecnología (CTO) le pide que elija un estilo arquitectónico para un nuevo sistema de procesamiento de transacciones financieras. Los requisitos clave son: (1) consistencia de datos absoluta (ACID); (2) capacidad de desplegar nuevas características de negocio de forma independiente y rápida; y (3) un equipo de desarrollo con experiencia mixta. Compare *Service-Based Architecture* y *Microservices Architecture* en función de estos tres requisitos. ¿Cuál de las dos es más adecuada para cumplir con el requisito de consistencia de datos y por qué? ¿Cómo se ve afectada la capacidad de despliegue independiente (requisito 2) en cada estilo? ¿Qué *trade-offs* debe aceptar el CTO si prioriza la consistencia sobre la agilidad de despliegue?

Para un sistema de transacciones financieras que exige consistencia ACID absoluta, con un equipo de experiencia mixta, la *Service-Based Architecture* (SBA) es más adecuada que la *Microservices Architecture*. A cambio, el CTO acepta unidades de despliegue más grandes, una base de datos compartida como punto de acoplamiento y menos elasticidad y capacidad evolutiva.

5.1. Comparación según los tres requisitos

Service-Based Architecture. Sus servicios de dominio de grano grueso suelen compartir una base de datos, por lo que una transacción completa se resuelve dentro de un solo servicio con *commit* y *rollback* (ver 5.2). Cada servicio se despliega de forma independiente, aunque como una unidad grande (ver 5.3). Respecto al equipo, obtiene 3 estrellas en simplicidad y 4 en costo: es uno de los estilos distribuidos más fáciles y baratos de implementar y ni siquiera requiere contenedores (Richards y Ford, 2020, cap. 13). Esto la hace accesible para un equipo de experiencia mixta.

Microservices Architecture. Cada servicio es dueño de sus propios datos, así que una operación que abarca varios servicios depende de consistencia eventual o de sagas (ver 5.2). Sus unidades de despliegue son muy pequeñas y el estilo alcanza la máxima capacidad evolutiva (ver 5.3). Respecto al equipo, obtiene solo 1 estrella en simplicidad y 1 en costo: definir la granularidad, la comunicación entre servicios y las sagas añade mucha complejidad de diseño y depuración (Richards y Ford, 2020, cap. 17), lo que resulta exigente para un equipo de experiencia mixta.

Las calificaciones de los autores para las características relevantes aparecen en la Figura 5 (1 = poco soportada, 5 = muy fuerte).

Architecture characteristic	Star rating
Partitioning type	Domain
Number of quanta	1 to many
Deployability	★★★★
Elasticity	★★★★★
Evolutionary	★★★★★
Fault tolerance	★★★★
Modularity	★★★★★
Overall cost	★
Performance	★★
Reliability	★★★★
Scalability	★★★★★
Simplicity	★
Testability	★★★★

Figure 17-13. Ratings for microservices

Architecture characteristic	Star rating
Partitioning type	Domain
Number of quanta	1 to many
Deployability	★★★★
Elasticity	★★
Evolutionary	★★★
Fault tolerance	★★★★
Modularity	★★★★
Overall cost	★★★★
Performance	★★★
Reliability	★★★★
Scalability	★★★
Simplicity	★★★
Testability	★★★★

Figure 13-9. Service-based architecture characteristics ratings

Figura 5: Calificaciones de las características arquitectónicas de la Microservices Architecture (izquierda) y de la Service-Based Architecture (derecha). *Nota. Fundamentals of Software Architecture: An Engineering Approach* (figs. 17-13 y 13-9), por M. Richards y N. Ford, 2020, O'Reilly Media.

5.2. ¿Cuál cumple mejor la consistencia de datos y por qué?

La SBA. Richards y Ford (2020, cap. 13) afirman que este estilo preserva las transacciones ACID mejor que cualquier otra arquitectura distribuida, precisamente por el grano grueso de sus servicios de dominio. Como un servicio contiene toda la lógica de un dominio y accede a una base de datos relacional, la operación entera se confirma o se revierte de forma atómica.

Trasladado a nuestro dominio, pensemos en una transferencia entre cuentas: debitar el origen, acreditar el destino, registrar el asiento contable y cobrar la comisión. En SBA, un servicio de dominio *Transacciones* ejecuta los cuatro pasos en una sola transacción de base de datos. En microservicios, si *Cuentas*, *Libro mayor* y *Comisiones* son servicios distintos, hace falta una saga: un mediador llama a cada parte y, si una falla, ordena deshacer las que ya tuvieron éxito (Richards y Ford, 2020, cap. 17; ver Figura 6). Mientras tanto existen estados intermedios visibles (*soft state*), lo que contradice el requisito de consistencia absoluta.

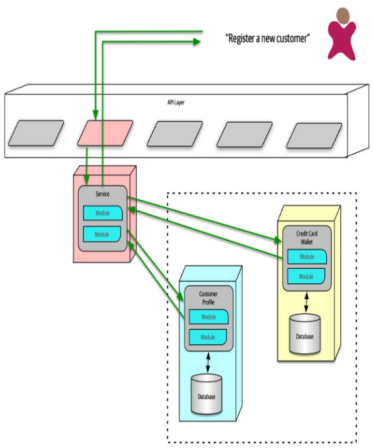


Figure 17-11. The saga pattern in microservices architecture

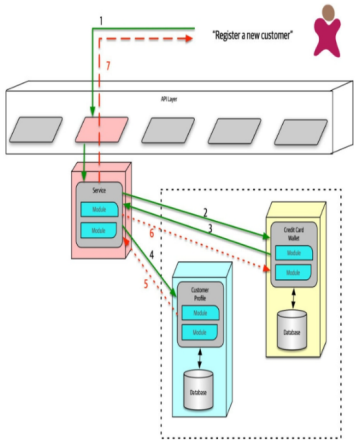


Figure 17-12. Saga pattern compensating transactions for error conditions

Figura 6: El patrón saga en microservicios (izquierda) y sus transacciones compensatorias ante un error (derecha). *Nota. Fundamentals of Software Architecture: An Engineering Approach* (figs. 17-11 y 17-12), por M. Richards y N. Ford, 2020, O'Reilly Media.

Los propios autores desaconsejan este camino: las transacciones entre servicios crean la peor

forma de connascencia dinámica, la connascencia de valor, y su consejo es corregir la granularidad en lugar de hacerlas (Richards y Ford, 2020, cap. 17). Además, las sagas pueden más que duplicar el trabajo de diseño, implementación y depuración (Richards y Ford, 2020, cap. 17).

Hay una condición: la SBA solo conserva ACID si cada operación financiera completa ocurre dentro de un único servicio de dominio. Si la interfaz o el API *gateway* orquestan dos servicios, también habría que recurrir a sagas y transacciones BASE (Richards y Ford, 2020, cap. 13).

5.3. Efecto sobre el despliegue independiente

En SBA cada servicio de dominio se despliega por separado, igual que una aplicación monolítica (un WAR, un EAR o un ensamblado), y un cambio frecuente queda aislado en su dominio (Richards y Ford, 2020, cap. 13). Sin embargo, la independencia es parcial por dos razones:

- **Tamaño de la unidad.** Un cambio pequeño exige probar y desplegar todo el servicio de grano grueso. En el ejemplo del libro, tocar la colocación de pedidos obliga a probar también el procesamiento de pagos, y desplegar más código implica más riesgo de romper algo (Richards y Ford, 2020, cap. 13).
- **Base de datos compartida.** Un cambio en el esquema puede afectar a todos los servicios. Si las entidades viven en una única biblioteca compartida, cualquier cambio obliga a redesplegar todos los servicios, usen o no la tabla modificada (Richards y Ford, 2020, cap. 13; ver Figura 7, izquierda). Además, servicios que comparten base de datos forman un solo cuanto arquitectónico (Richards y Ford, 2020, caps. 7 y 13).

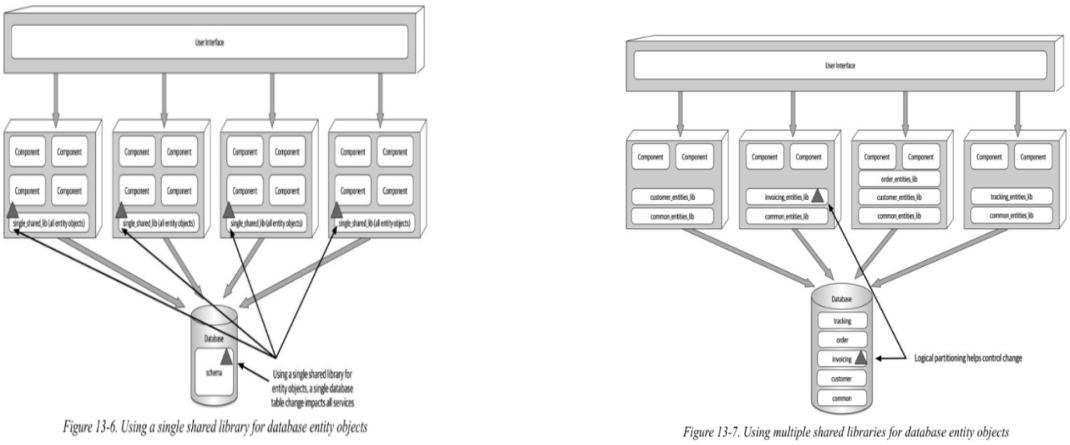


Figura 7: Una única biblioteca compartida de entidades (izquierda) frente a bibliotecas compartidas federadas por dominio de datos (derecha) en una Service-Based Architecture. *Nota. Fundamentals of Software Architecture: An Engineering Approach* (figs. 13-6 y 13-7), por M. Richards y N. Ford, 2020, O'Reilly Media.

En microservicios, cada servicio es pequeño y de propósito único, de modo que un cambio solo lo afecta a él, y el sistema tiene muchos cuantos (Richards y Ford, 2020, cap. 17). Pero esa ventaja depende de la automatización y la madurez DevOps (Richards y Ford, 2020, cap. 17), y las sagas vuelven a acoplar en tiempo de ejecución a los servicios que participan en ellas.

5.4. *Trade-offs* que acepta el CTO al priorizar la consistencia

Elegir SBA es, en términos de Richards y Ford (2020, cap. 13), aceptar que los servicios gruesos dan mejor integridad de datos a cambio de cambios más costosos de probar y desplegar. En concreto, el CTO acepta:

- **Despliegues más grandes y con más riesgo.** Cada versión incluye todo un dominio, con más pruebas de regresión por cambio.
- **Coordinación de cambios de esquema.** La base de datos compartida es el principal punto de acoplamiento entre servicios.
- **Menos elasticidad y escalabilidad.** Escalar un servicio grueso replica más funcionalidad y consume más recursos (Richards y Ford, 2020, cap. 13).
- **Menor capacidad evolutiva y menos cuantos,** por lo que las partes del sistema comparten las mismas características operativas.
- **Menos libertad tecnológica.** Los servicios suelen compartir pila y bibliotecas, mientras que los microservicios permiten entornos políglotas (Richards y Ford, 2020, cap. 17).

Estos costos pueden reducirse sin renunciar a ACID:

- Particionar la base de datos lógicamente y usar bibliotecas compartidas federadas, una por dominio de datos, para que un cambio de tabla solo afecte a los servicios de ese dominio (Richards y Ford, 2020, cap. 13; ver Figura 7, derecha).
- Escalar solo los servicios con más carga, como hace el ejemplo de reciclaje con *Quoting* e *ItemStatus* (Richards y Ford, 2020, cap. 13).

A cambio de esos costos, el CTO obtiene integridad transaccional nativa y mayor fiabilidad, porque hay menos tráfico de red y menos transacciones distribuidas (Richards y Ford, 2020, cap. 13). También elige un estilo acorde a un equipo mixto: el capítulo 18 advierte que los estilos que dependen de prácticas ágiles maduras presentan dificultades a las organizaciones que no las tienen (Richards y Ford, 2020, cap. 18). Por eso recomendamos la SBA, con cada operación financiera completa dentro de un único servicio de dominio.

1.6. Pregunta 6: El legado de SOA y su influencia en arquitecturas modernas

La *Service-Oriented Architecture* (SOA) con orquestación a menudo es criticada por su complejidad y por crear un cuello de botella central (el motor de orquestación). Sin embargo, algunos de sus principios, como la reutilización de servicios, siguen siendo relevantes. SOA también admite coreografía, no solo orquestación. Analice cómo la *Microservices Architecture* aborda el problema de la reutilización de una manera fundamentalmente diferente. ¿Cómo maneja la reutilización de lógica de negocio (p. ej., validación de clientes) frente a la reutilización operativa (p. ej., *logging*, monitoreo)? ¿En qué escenarios un arquitecto podría todavía considerar un enfoque de orquestación (como un mediador local en microservicios o un *orchestration engine* en SOA) a pesar de sus desventajas? Proporcione un ejemplo de un flujo de trabajo complejo que se beneficiaría de la orquestación explícita.

6.1. El problema de la reutilización en SOA

En la SOA orientada a orquestación, la reutilización empresarial era la filosofía dominante: servicios empresariales de grano fino y compartidos, combinados por un motor central. En el ejemplo de la aseguradora del libro, un servicio canónico *Customer* usado por todas las divisiones logró reutilización, pero cada cambio en él se propagaba a los demás servicios y obligaba a despliegues coordinados y pruebas integrales (Richards y Ford, 2020, cap. 16; ver Figura 8).

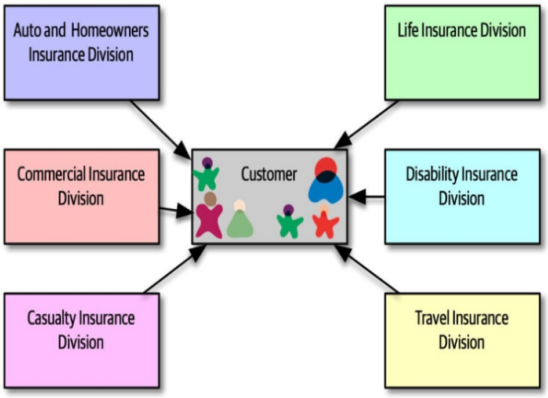


Figure 16-4. Building canonical representations in service-oriented architecture

Figura 8: Representación canónica del servicio Customer compartido por todas las divisiones de una aseguradora en SOA. *Nota. Fundamentals of Software Architecture: An Engineering Approach* (fig. 16-4), por M. Richards y N. Ford, 2020, O'Reilly Media.

6.2. Cómo lo aborda la arquitectura de microservicios

El punto de partida es la Primera Ley de la Arquitectura de Software: el costo de la reutilización es el acoplamiento. Como el objetivo principal de los microservicios es el alto desacoplamiento, el estilo prefiere la duplicación a la reutilización, aunque trata de forma distinta lo de dominio y lo operativo (Richards y Ford, 2020, cap. 17).

Reutilización de lógica de negocio (p. ej., validación de clientes). Cada servicio incluye todo lo que necesita para operar, desde sus clases hasta su esquema de base de datos, y nunca se acopla a las clases o datos de otro *bounded context*. Así cada contexto define solo lo que necesita (Richards y Ford, 2020, cap. 17). Por tanto, no existe un servicio *ValidarCliente* común: en un banco, *Alta de clientes* valida identidad y documentos, *Créditos* aplica sus reglas de solvencia y *Pagos* solo comprueba que la cuenta esté activa.

Si un dato necesita una única fuente de verdad, como el estado del cliente, hay dos opciones. Se puede designar un dominio como dueño del dato y consultarlo, o distribuirlo mediante replicación o caché (Richards y Ford, 2020, cap. 17). El costo de duplicar es tener que actualizar una regla en varios servicios. El estilo lo acepta para evitar el efecto dominó del *Customer* canónico de SOA.

Reutilización operativa (p. ej., *logging*, monitoreo). Lo operativo sí se beneficia del acoplamiento, porque es difícil garantizar que cada equipo implemente y actualice por su cuenta el monitoreo (Richards y Ford, 2020, cap. 17). Con el patrón *sidecar*, el *logging*, el monitoreo y los *circuit breakers* viven en un componente aparte dentro de cada servicio. Un equipo de infraestructura lo actualiza una sola vez y todos los servicios reciben el cambio. Los *sidecars* se conectan formando una *service mesh*, que ofrece una consola para controlar estas preocupaciones de forma global (Richards y Ford, 2020, cap. 17; ver Figura 9).

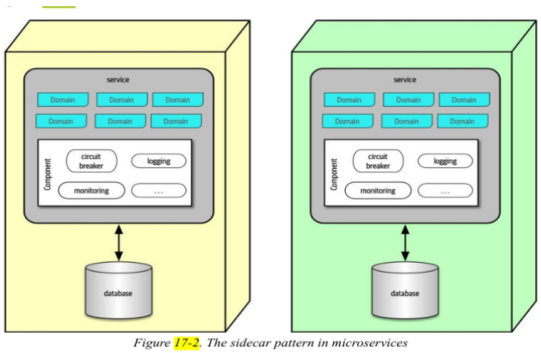


Figure 17-2. The sidecar pattern in microservices

Figura 9: El patrón sidecar en microservicios: las preocupaciones operativas (*circuit breaker*, *logging*, monitoreo) en un componente separado dentro de cada servicio. *Nota. Fundamentals of Software Architecture: An Engineering Approach* (fig. 17-2), por M. Richards y N. Ford, 2020, O'Reilly Media.

A diferencia de SOA, donde los servicios de infraestructura se invocaban a través del bus de orquestación, el *sidecar* viaja con cada servicio y no interviene en la lógica de negocio. Se gana uniformidad operativa sin crear un punto central de acoplamiento de dominio.

6.3. Escenarios en los que todavía se considera la orquestación

Un arquitecto podría considerar un enfoque de orquestación cuando un proceso de negocio necesita coordinar explícitamente varios servicios, controlar las dependencias entre sus pasos, integrar sistemas heterogéneos o gestionar fallos que afectan al resultado del conjunto. En estos escenarios, concentrar la coordinación puede resultar preferible a distribuirla entre numerosos participantes, a pesar de sus desventajas: un mayor acoplamiento (Richards y Ford, 2020).

Ampliando la primera idea, la orquestación puede ser adecuada para procesos de negocio complejos con dependencias entre servicios, o sea, cuando terminar un proceso exige coordinar numerosas operaciones y determinar qué debe ejecutarse en función de los resultados anteriores. El capítulo 16 del libro explica que el motor vincula las implementaciones de los servicios y define cómo se relacionan para realizar un comportamiento de negocio. Además, en el capítulo siguiente (sobre microservicios) los autores reconocen que algunos flujos están inherentemente acoplados. Para estos casos, un mediador dedicado puede concentrar la complejidad de coordinación y evitar que cada servicio tenga que comprender el proceso completo. El beneficio consiste en representar explícitamente las dependencias del negocio y limitar dónde se administra su coordinación.

Otro escenario es la integración de sistemas existentes que deben participar en un mismo proceso. El capítulo 16 señala que el motor de orquestación puede actuar como centro de integración entre código desarrollado por la organización, aplicaciones comerciales y sistemas heredados. También puede realizar transformaciones de mensajes y establecer la correspondencia entre los servicios que intervienen. Por tanto, un motor de orquestación ofrece un lugar explícito para definir la colaboración en una organización que necesite combinar capacidades de sistemas que no puede sustituir inmediatamente (teniendo en cuenta que convertir ese motor en intermediario de todas las llamadas crea un punto de acoplamiento significativo) (Richards y Ford, 2020).

La orquestación también puede considerarse cuando varios servicios deben participar en una operación y es necesario decidir qué hacer si alguno falla después de que otros hayan terminado su trabajo. El capítulo 17 desarrolla, usando sagas, el siguiente escenario: un mediador coordina las operaciones, registra sus resultados y solicita acciones compensatorias cuando una etapa falla. Sin embargo, los autores recomiendan utilizar este mecanismo con moderación y revisar primero la granularidad, porque una necesidad constante de transacciones entre servicios puede revelar límites incorrectamente definidos. Además, advierten que definir los límites de las transacciones distribuidas se vuelve progresivamente complejo. Por consiguiente, la orquestación transaccional

es comprensible cuando existen razones para mantener separados los servicios, por ejemplo, requisitos arquitectónicos muy diferentes pero igual necesitan coordinación.

En una situación donde el primer servicio que recibe una solicitud se ve obligado a llamar a muchos otros y asumir funciones de mediación (además de sus responsabilidades de dominio) un mediador local permite separar la coordinación del flujo de las funciones específicas de los participantes. Así se evita un aumento considerable de complejidad al impedir que un servicio de negocio se convierta en un coordinador implícito. La diferencia respecto al motor central de la variante SOA es su alcance ya que coordina un proceso determinado, en lugar de convertirse en el intermediario obligatorio de toda la organización (Richards y Ford, 2020).

Ejemplo de un flujo de trabajo que se beneficiaría de la orquestación explícita. Un ejemplo sería la incorporación de un empleado que necesita formalizar la relación laboral, preparar su registro en nómina, tramitar beneficios y habilitar sus accesos corporativos. Se supone que estas capacidades pertenecen a sistemas distintos (algunos heredados) y que sus resultados deben coordinarse para determinar el estado global de la incorporación. Su secuencia sería la siguiente:

Etapa	**Acción del orquestador de incorporación**	**Responsabilidad del sistema participante**
1. Inicio	Recibe la solicitud e inicia el seguimiento del proceso.	Gestión de Contratación verifica y registra el alta laboral mediante sus propias reglas.
2. Preparación de nómina	Solicita la creación del registro salarial cuando el alta está confirmada.	Nómina valida la información que necesita y prepara el registro correspondiente.
3. Tramitación de beneficios	Solicita evaluar la elegibilidad e iniciar las inscripciones aplicables.	Beneficios aplica sus propias reglas de elegibilidad e inscripción.
4. Habilitación de accesos	Solicita los accesos autorizados cuando se cumplen las condiciones previas.	Identidad y Accesos crea o habilita las cuentas correspondientes.
5. Cierre	Evalúa los resultados y marca la incorporación como completada o pendiente de resolución.	Cada participante conserva la responsabilidad sobre sus datos y operaciones.

La utilidad del orquestador está en que la secuencia y las condiciones del proceso quedan en un componente dedicado. Nómina no necesita dirigir la creación de accesos y Beneficios no necesita conocer todas las etapas de incorporación. Esta distribución sigue la idea de concentrar la coordinación en un servicio para que los demás resulten menos afectados por la complejidad del flujo (Richards y Ford, 2020).

También se consideró otro escenario para analizar su flujo: el alta laboral y la preparación de nómina se completan, pero falla la habilitación de accesos. El orquestador puede conservar el proceso en un estado pendiente e identificar la etapa que no se completó, sin presentar la incorporación como finalizada. Si este proceso exigiera cancelar acciones anteriores, el orquestador podría solicitar compensaciones específicas: como cancelar una inscripción provisional en beneficios o desactivar una cuenta creada durante el intento.

Este flujo se beneficia de la orquestación explícita porque combina integración de sistemas, dependencias entre etapas y tratamiento conjunto de resultados. No obstante, el coordinador debería limitarse a la incorporación y no convertirse en el paso obligatorio de todas las operaciones de recursos humanos. Así se acepta un acoplamiento localizado para representar una necesidad real del negocio, sin reproducir innecesariamente el acoplamiento que ocurre en SOA.

Referencias

[Ford et al.(2017)] Ford, N., Parsons, R., y Kua, P. (2017). *Building Evolutionary Architectures: Automated Software Governance*. O'Reilly Media.

[Ford et al.(2021)] Ford, N., Richards, M., Sadalage, P., y Dehghani, Z. (2021). *Software Architecture: The Hard Parts*. O'Reilly Media.

[Gilbert y Lynch(2002)] Gilbert, S., y Lynch, N. (2002). Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News, 33*(2), 51–59.

[Richards y Ford(2020)] Richards, M., y Ford, N. (2020). *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media.

Declaración de Uso de la Inteligencia Artificial

Para la revisión y el desarrollo de este seminario, se utilizaron como herramientas de apoyo los modelos de inteligencia artificial ChatGPT (GPT-6) y Gemini (flash 3), este último empleado para la elaboración y corrección de los elementos en LaTeX. No obstante, el equipo de trabajo asume la responsabilidad sobre la veracidad de la información, la adecuación de las referencias y la integridad del contenido final presentado.